

Introduction to Algorithm

기초 알고리즘

OT, 수업 소개

과목 소개

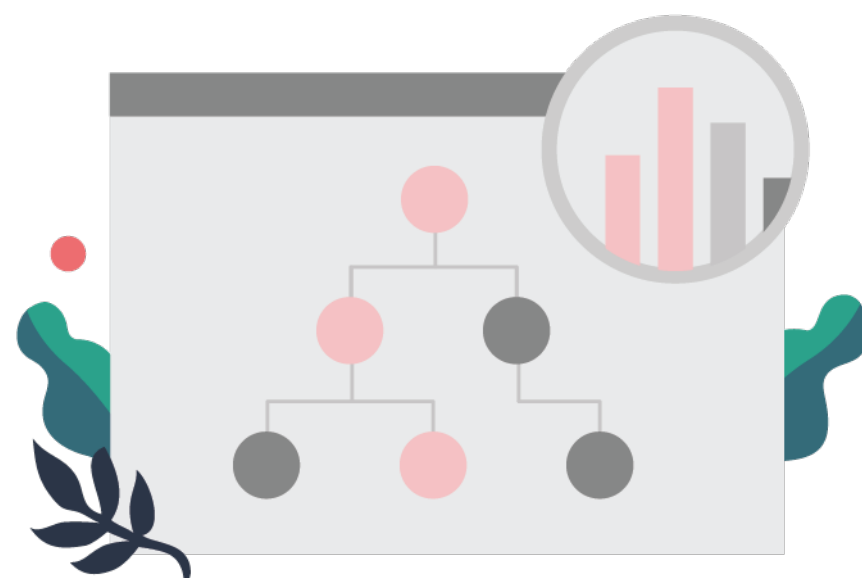


컴퓨터 공학의 기본 커리큘럼



1. 프로그래밍 언어

C / C++
Python
Matlab



2. 자료구조

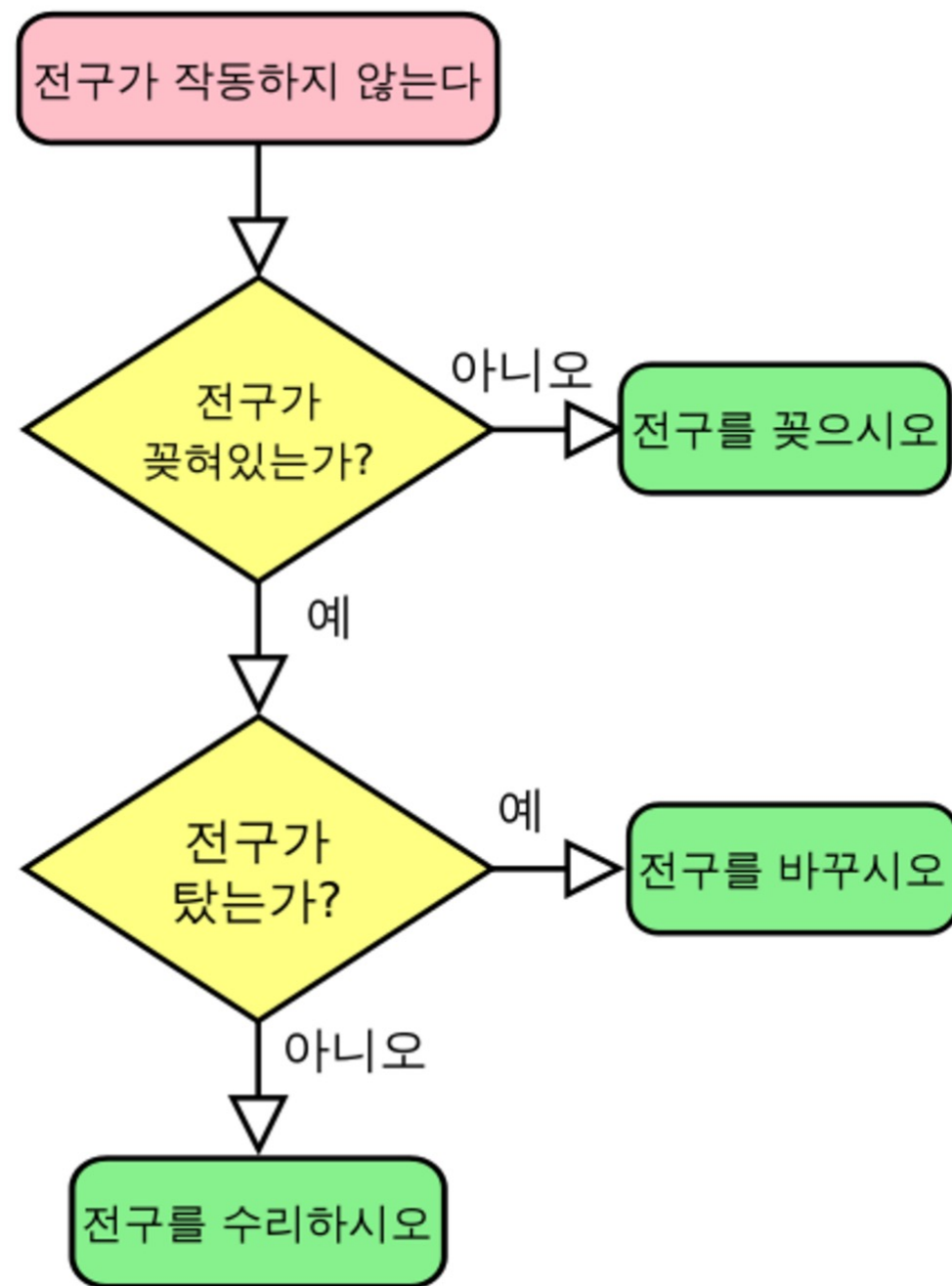
Stack
Queue
Tree



3. 알고리즘

Brute-Force
Divide & Conquer
Dynamic Programming

알고리즘 = 문제를 해결하는 방법



```
def fixBulb(bulb):  
    if not bulb.isEmpty():  
        bulb.create()  
  
    elif bulb.isBurnt:  
        bulb.change()  
  
    else:  
        bulb.fix()
```

계산을 통하여
해결할 수 있는 문제를
해결하는 방법



들여가기에 앞서



스마트폰의 연락처를 수기로 노트에 적는다고 가정해 보자



들어가기에 앞서

사람들의 전화번호를 아는 대로 선착순으로 저장

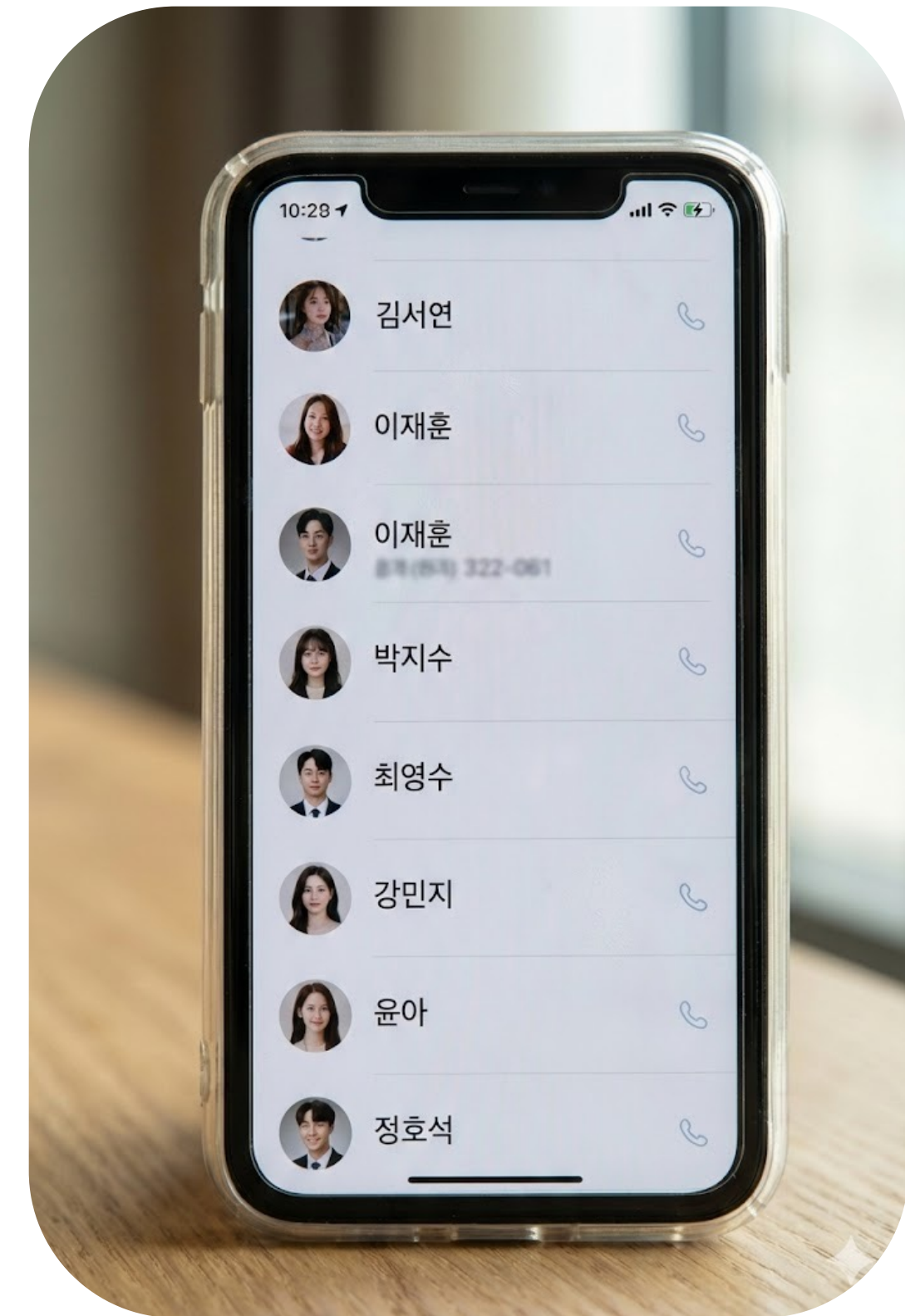
전화번호의 탐색 시간이 오래 걸림

자음 순으로 정렬(ex. ㄱ, ㄴ, ㄷ, ㄹ...)

중간에 새로운 전화번호 저장 시, 시간이 오래 걸림

자음 순으로 정렬하되, 여유 공간을 생성

사용하지 않는 여유 공간만큼 낭비

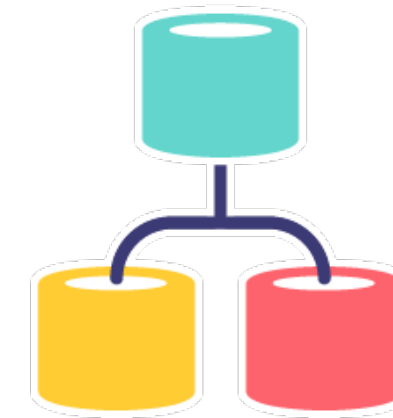




자료구조와 알고리즘의 관계

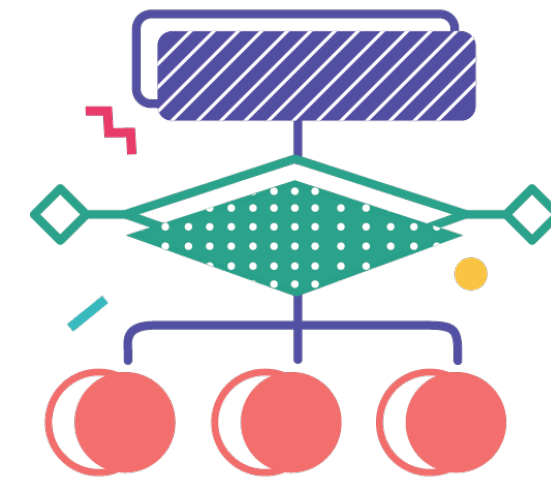
전화번호부의 저장 구조 → **자료구조**

데이터를 저장하고 관리하는 방식(데이터의 흐름과 저장)



전화번호를 찾는 방법 → **알고리즘**

문제를 해결하는 방법(시간과 공간 효율성)



자료구조가 달라지면 알고리즘도 달라짐



[예제] k번째 숫자 찾기

n개의 숫자 중 "지금까지 입력된 숫자들 중에서 k번째 숫자"는 ?
(단, $1 \leq k \leq n \leq 100$)

입력 예시

10 3

1 9 8 5 2 3 5 6 2 10

순서대로 n, k, 숫자들이 입력됨



[예제] k번째 숫자 찾기

n개의 숫자 중 "지금까지 입력된 숫자들 중에서 k번째 숫자"는 ?
(단, $1 \leq k \leq n \leq 100$)

입력 예시

10 3

1 9 8 5 2 3 5 6 2 10

문제를 해결하는 방법

1. 숫자 하나를 입력받는다
2. 지금까지 받은 숫자들을 정렬한다
3. k번째로 작은 숫자를 출력한다

과정 구성

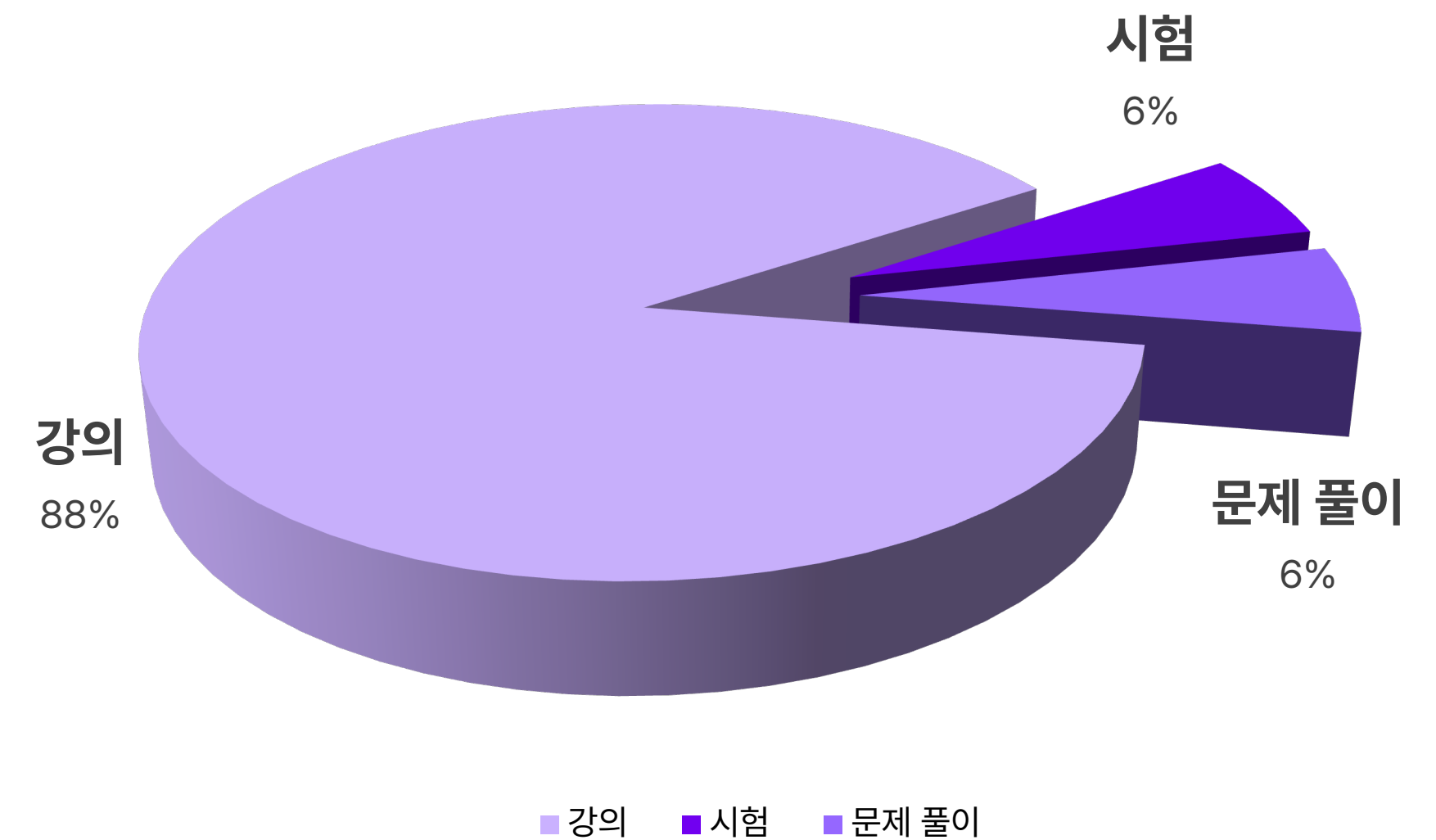
과정 구성

주 2회(차시), 총 16주 32차시

1차시 = 이론 강의 + 퀴즈 / 실습

중간고사 (8주차) + 기말고사 (16주차)

과제 총 6회 부여



주차별 수업 내용 (1~3주차)

주차	차시	강의주제	강의 내용
1주차 알고리즘 및 기본 문제 소개	1	OT, 수업 소개	기초 알고리즘 수업 운영 방향 소개
	2	알고리즘의 개념	알고리즘의 정의와 특성 학습
2주차 알고리즘 분석을 위한 수학	3	알고리즘 분석을 위한 수학 (1)	복잡도 분석 개요
	4	알고리즘 분석을 위한 수학 (2)	수학적 분석을 통한 효율적 프로그램 작성
3주차 Divide-and- Conquer 방법	5	Divide and Conquer 개요	문제 분할 전략과 재귀 기반 설계
	6	Merge Sort와 Quick Sort	Merge Sort & Quick Sort 구현 및 분석

주차별 수업 내용 (4~6주차)

주차	차시	강의주제	강의 내용
4주차 Divide-and-Conquer 방법	7	다양한 정렬 알고리즘	정렬 알고리즘 비교
	8	Binary Search	Binary Search 및 기타 응용
5주차 그리디 (Greedy 방법)	9	Greedy 알고리즘 개요	탐욕적 선택 전략 개요
	10	Greedy 알고리즘 구현	대표 예제 탐구 및 구현
6주차 그리디 (Greedy 방법)	11	Greedy 알고리즘 심화	Greedy 알고리즘 심화문제 풀이
6주차 다이나믹 프로그래밍 (Dynamic Programming)	12	다이나믹 프로그래밍 개요 1	DP 개요 및 적용 전략 1

주차별 수업 내용 (7~9주차)

주차	차시	강의주제	강의 내용
7주차 다이나믹 프로그래밍 (Dynamic Pro gramming)	13	다이나믹 프로그래밍 개요 2	DP 적용 전략 2
	14	다이나믹 프로그래밍 기초 예제	기초 DP 문제 해결
8주차 중간고사	15	중간고사 대비 문제 풀이	중간고사 대비 실전 문제 연습
	16	중간고사	중간고사 평가
9주차 다이나믹 프로그래밍 (Dynamic Pro gramming)	17	다이나믹 프로그래밍 심화 1	대표 DP 유형 정리 1
	18	다이나믹 프로그래밍 심화 2	대표 DP 유형 정리 2

주차별 수업 내용 (10~12주차)

주차	차시	강의주제	강의 내용
10주차 그래프 (Graph) 이론	19	그래프 이론 기초	그래프의 정의와 표현 방법
	20	그래프 알고리즘 (1)	최단 경로 알고리즘
11주차 그래프 (Graph) 알고리즘	21	그래프 알고리즘 (2)	최소 신장 트리
	22	그래프·트리 알고리즘 응용	위상 정렬, LCA
12주차 네트워크 플로우 (Network Flow) 문제	23	네트워크 플로우 개념	최대 유량 문제와 개요
	24	네트워크 플로우 응용	응용 사례 탐색

주차별 수업 내용 (13~15주차)

주차	차시	강의주제	강의 내용
13주차 백트래킹 (Backtracking)	25	Backtracking 개요	완전 탐색과 백트래킹의 차이
	26	Backtracking 구현 및 실습	N-Queen, 순열 생성 등
14주차 Tractable and Intractable 문제	27	Tractable vs Intractable 개념	계산 가능성과 복잡성
	28	Intractable 문제 사례 탐색	NP 완전 문제 소개
15주차 Tractable and Intractable 문제 및 근사해결 방법	29	Intractable 문제 해결 접근	근사 알고리즘 및 휴리스틱
	30	Intractable 문제 해결 전략	현실적인 해법 고찰

주차별 수업 내용 (16주차)

주차	차시	강의주제	강의 내용
16주차 기말고사	31	기말고사 대비 문제 풀이	최종 복습 및 실전 문제 풀이
	32	기말고사	기말고사 평가

Introduction to Algorithm

기초 알고리즘

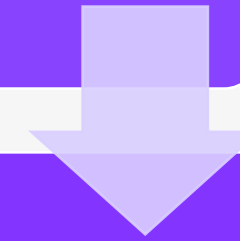
생성 AI와 바이트코딩



변화하는 패러다임: 공동 개발자로의 AI

기계어, 어셈블리

- 가장 저수준 (Low Level) 의 프로그래밍 언어.
- 컴퓨터의 모든 연산을 수동으로 통제.



고급 프로그래밍 언어

- 저수준 언어를 한 단계 추상화 한 고수준 (High Level) 언어.
- 한 단계 추상화를 거쳐 개발자가 알고리즘 등 기능 구현의 본질에 더 집중할 수 있도록 한다.



통합 개발 환경 (IDE)

- 기능 개발을 넘어서 프로젝트를 관리하는 개발 환경
- 개발자가 전체 프로젝트를 한 눈에 보고 관리할 수 있도록 하여, 문제 해결의 본질에 한 단계 더 집중하도록 한다.



생성형 AI

- AI가 반복적인 코드 작성 작업을 자동화함에 따라, 개발자는 문제 해결, 가치 창출을 위한 창의적, 전략적인 업무에 집중할 수 있다.
- 개발자의 역할이 프로그램을 구현하는 것에서 문제 해결을 위해 AI의 생성물을 감독하고 통합하는 방향으로 진화.

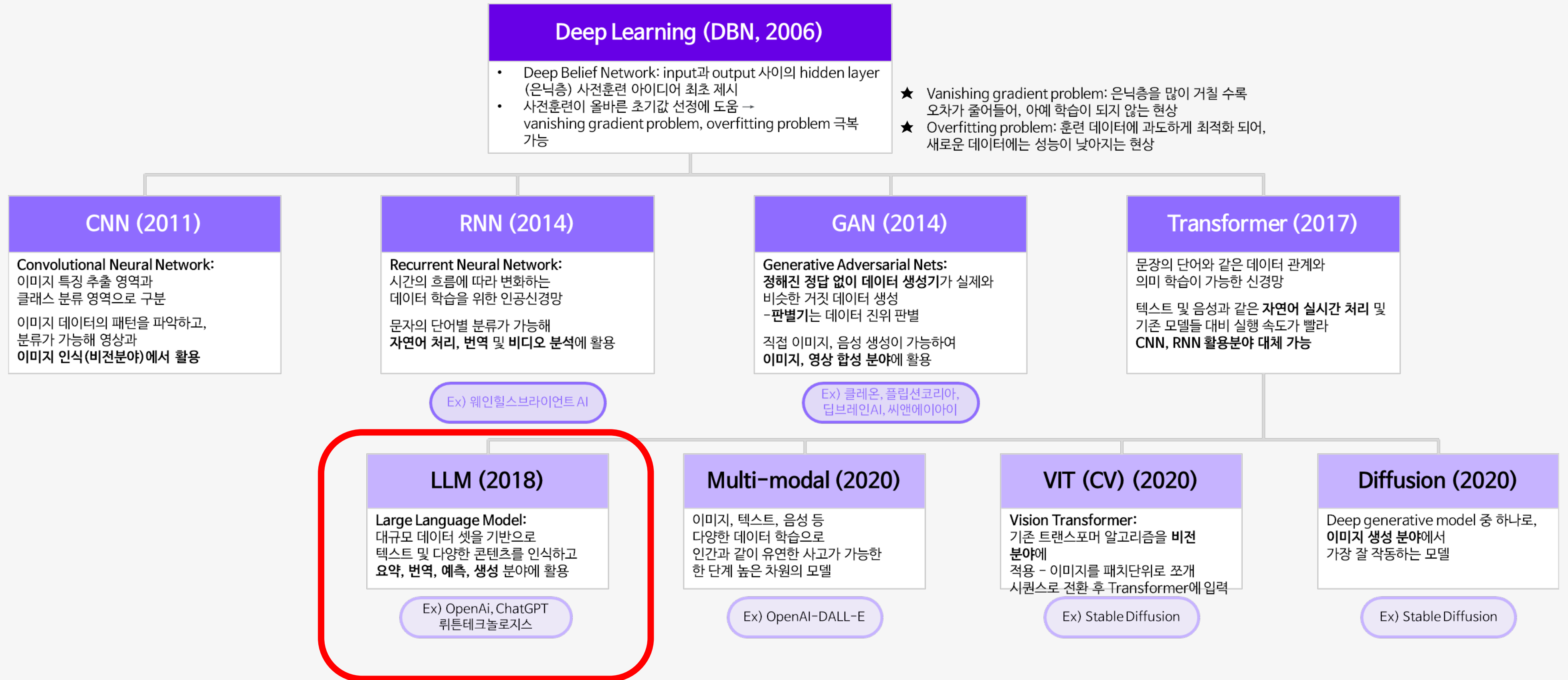


생성형 AI

텍스트, 이미지, 코드 혹은 다른 미디어를
생성할 수 있는 인공지능

방대한 코드 데이터로부터 프로그래밍 패턴, 구문, 추상화된 로직 까지
학습하여 주어진 요구사항에 맞는 코드를 생성할 수 있다.

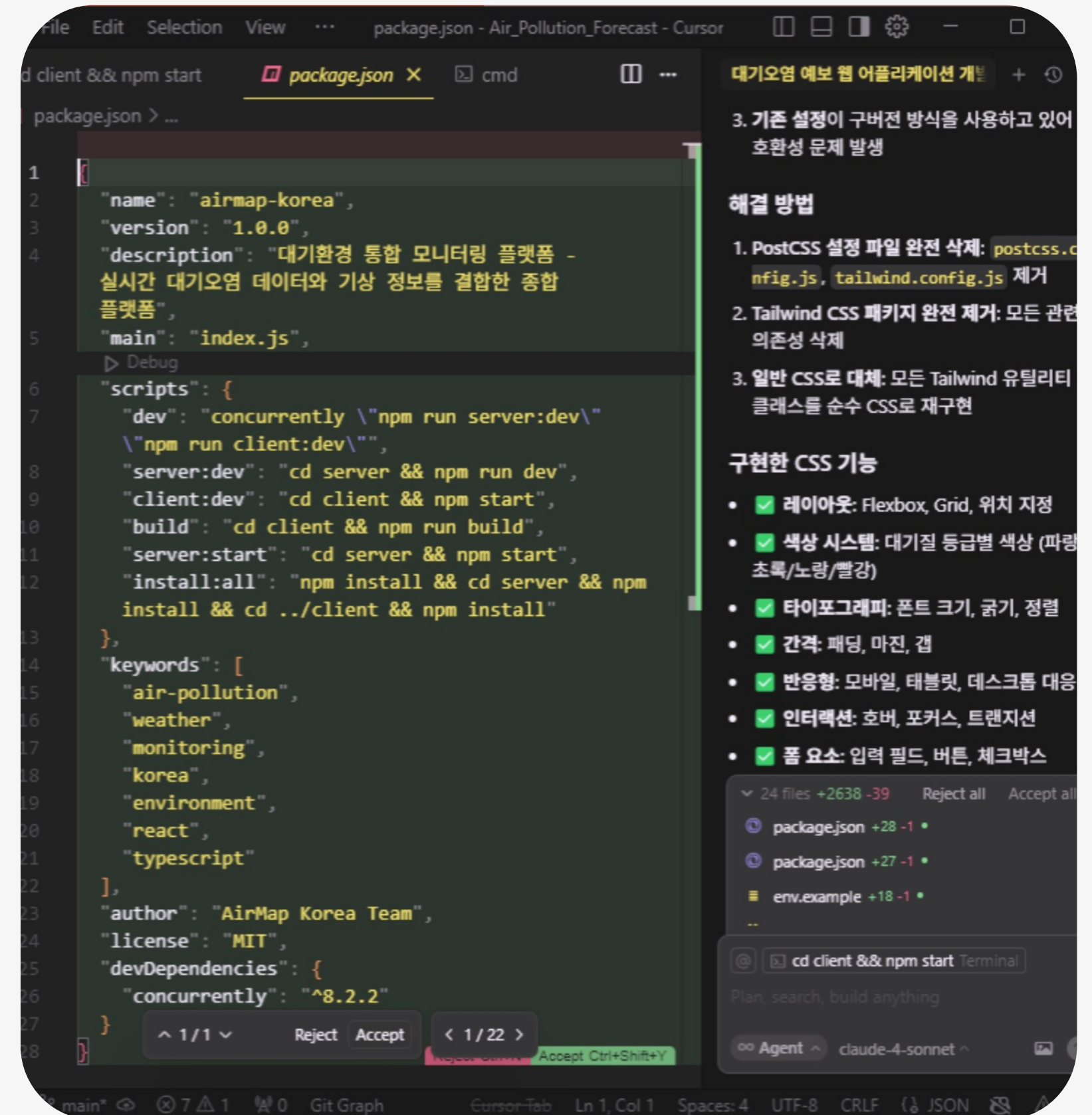
트랜스포머 알고리즘(2017)





거대 언어 모델 (LLM)

- 언어 모델(LM): 인간의 언어 능력을 모사하는 모델, 현재 존재하는 문장 내의 단어를 기반으로 앞으로 나올 단어를 예측
- 거대 언어 모델(LLM): 언어 모델 중 그 규모가 매우 큰 모델. 대규모 데이터셋을 바탕으로 텍스트 뿐만 아니라 다양한 콘텐츠를 인식하고, 요약, 번역, 예측, **생성** 분야에 활용 가능.
- 방대한 코드 데이터를 학습한 LLM은 데이터로부터 프로그래밍 패턴, 구문, 추상화된 로직 까지 학습하여 주어진 요구사항에 맞는 **코드를 생성할 수 있다.**



거대 언어 모델 claude-4-sonnet 으로
생성한 코드 및 관련 설명

생성형 AI 서비스

생성형 AI 서비스



Claude



ChatGPT



Gemini



DeepSeek

생성형 AI 기반 개발 도구



Antigravity



Cursor



Claude Code

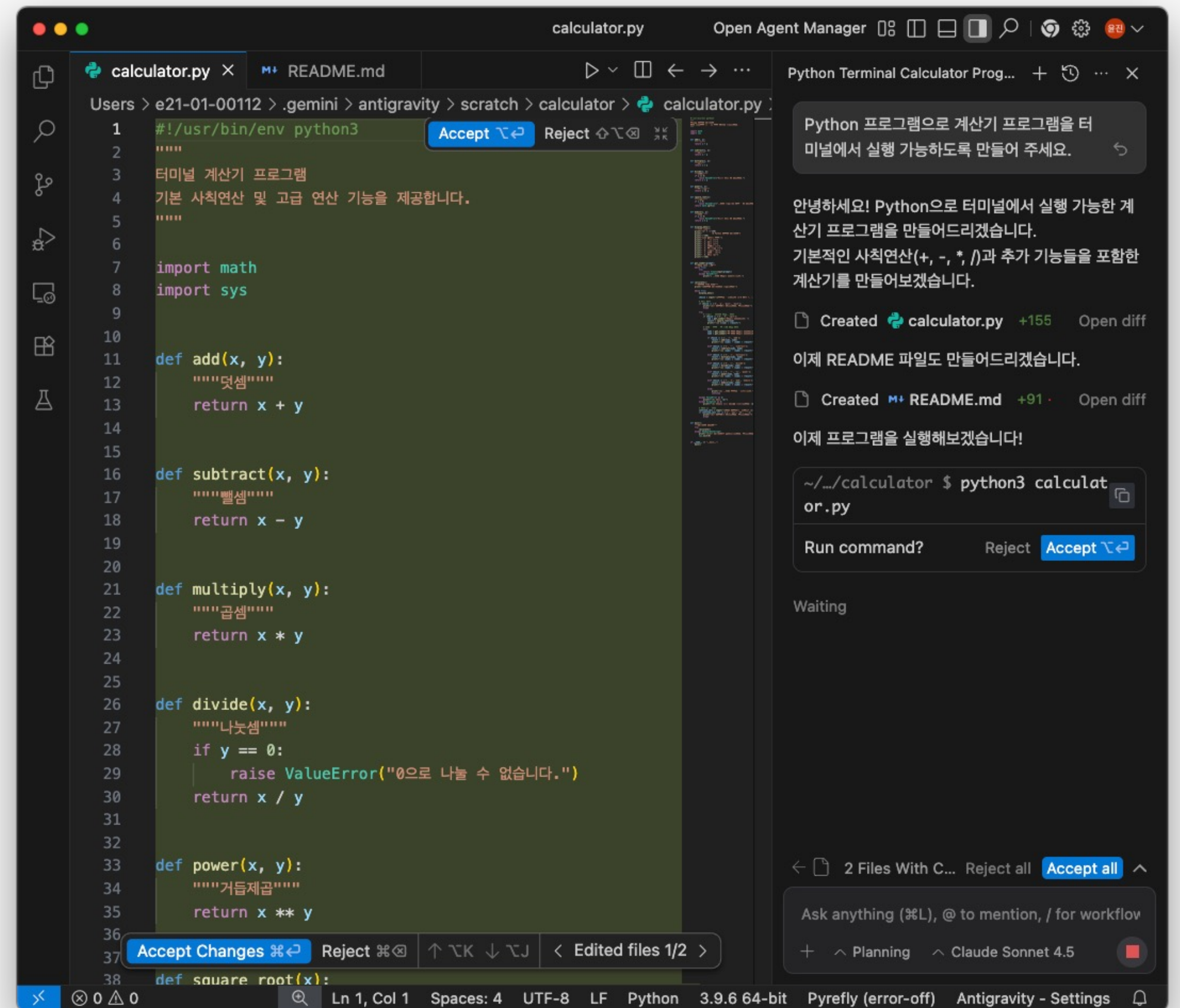


Github Copilot

생성형 AI 기반 개발 도구 - Antigravity

Google에서 개발한 AI 에이전트 기반 통합 개발 환경(IDE)으로,
개발자가 **자연어 지시**를 내리면 AI가 코딩, 테스트, 브라우징 등
반복적인 개발 작업을 자율적으로 수행하게 해주는 도구

편집기·터미널·브라우저를 오가며 계획 → 구현 → 실행 → 검증까지
한 흐름으로 처리





Vibe Coding

개발자의 직관(Vibe)과 창의성, 그리고 **생성형 AI의 지원을 결합하여**
코드를 작성하는 코딩 패러다임

개발 패러다임 비교

전통적 개발

수동으로 한 줄씩 코딩

개발자가 완전한 제어권을
가짐

핵심 기술: 언어 구문, 알고
리즘 지식

AI 활용 개발

AI가 코드를 제안/완성

개발자가 AI의 제안을 검토/
수정

핵심 역량: 알고리즘 지식
기반 AI 제안 검토 능력

Vibe Coding

개발자가 자연어로 AI에게
의도를 지시

AI가 주도적으로 코드 생성,
수정

핵심 역량: 프롬프트 엔지니
어링, AI 결과물 검토 능력

전통적 개발 절차 (예시)

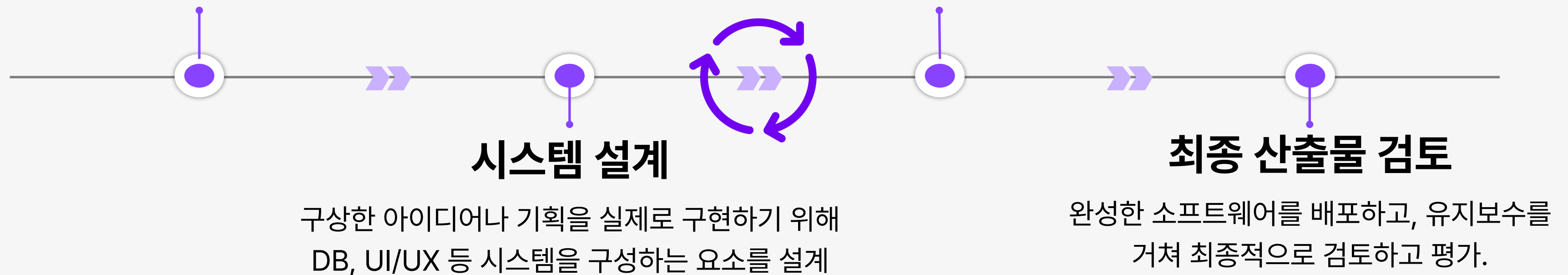
아이디어를 구상하고, 시스템을 설계하고, 코드로 구현하고, 테스트를 거쳐 배포하는 과정

아이디어 구상 (기획)

고객의 요구사항이나 주어진 문제를 해결하기 위한 핵심 아이디어나 컨셉을 구상

코드 구현, 테스트 및 디버깅

설계 명세를 바탕으로 코드를 구현하고, 다양한 테스트를 통해 올바르게 구현되었는지 확인
필요에 따라 **이전 과정으로 돌아가며 반복적으로 개선**



Vibe Coding 절차 (예시)

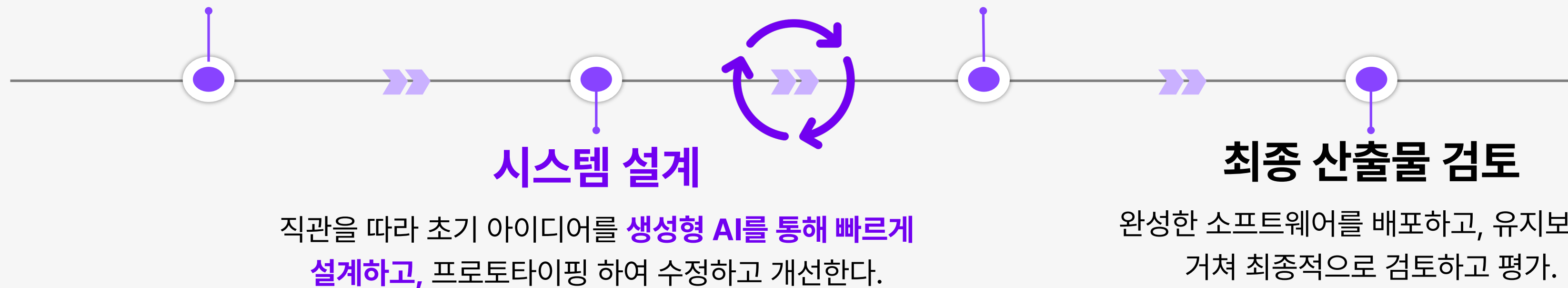
전통적 개발 절차와 흐름은 유사하지만, 각 과정을 **AI를 활용해 크게 효율화**

아이디어 구상 (기획)

고객의 요구사항이나 주어진 문제를 해결하기 위한 핵심 아이디어나 컨셉을 구상하고, **생성형 AI를 활용해 구체화 한다.**

코드 구현, 테스트 및 디버깅

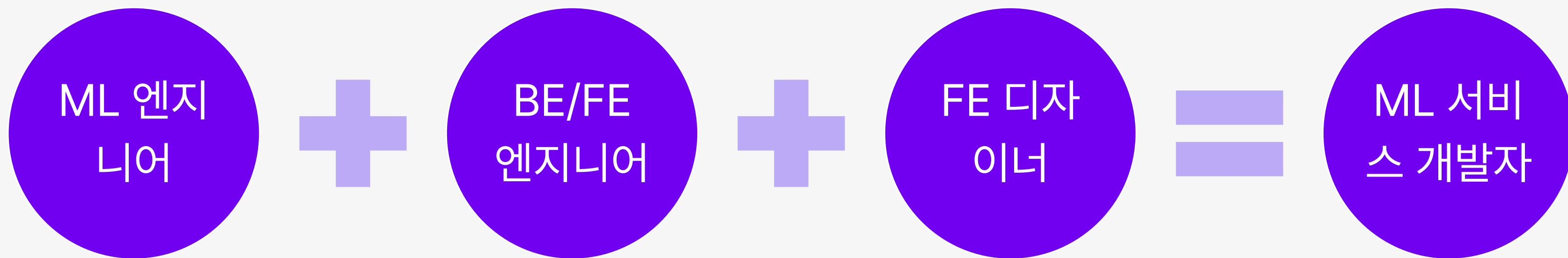
생성형 AI를 활용하여 코드를 빠르게 구현하고, 다양한 테스트를 통해 올바르게 구현되었는지 확인한다. 필요에 따라 **이전 과정으로 돌아가며 반복적으로 개선**



프로그래밍 생산성 향상

전통적 개발 절차를 **AI를 활용해 효율화** 할 수 있게 되면서, 프로젝트 규모에 따라 한 사람이 전통적인 개발 직무 여럿을 **혼자서 아우르는 것도 가능하다**.

예시로, **작은 프로젝트에서는** 머신러닝(ML) 엔지니어 직무의 개발자가 Vibe Coding을 활용해 백엔드(BE), 프론트엔드(FE) 구조를 구현하고, Figma MCP를 활용해 FE 디자인까지 완료하여 **혼자서 머신러닝 서비스를 개발할 수 있다**.



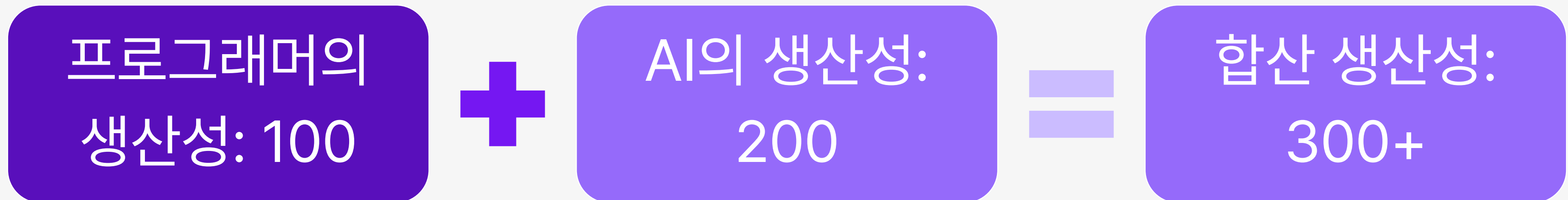
	ML 엔지니어링	BE/FE 엔지니어링	FE 디자인	합산
필요 생산성	100	100	100	300
프로그래머의 생산성	90	5	5	100
AI의 생산성	10	95	95	200

Note. 임의로 작성한 수치로, 정량적으로 측정한 값이 아닙니다.

프로그래밍 생산성 향상

여러 직무를 융합하는 것 뿐만 아니라, **본래 각자의 직무에서의 생산성을 크게 끌어올리는 것도 가능하다.**

이를 반영하여, 일부 기업에서는 개발자 공채에 생성형 AI를 비롯한 인공지능을 활용하는 것을 허가하기도 한다.



Note. 임의로 작성한 수치로, 정량적으로 측정한 값이 아닙니다.

Vibe Coding 활용 개발 생산성 향상 예시

코드 생성 및 자동 완성

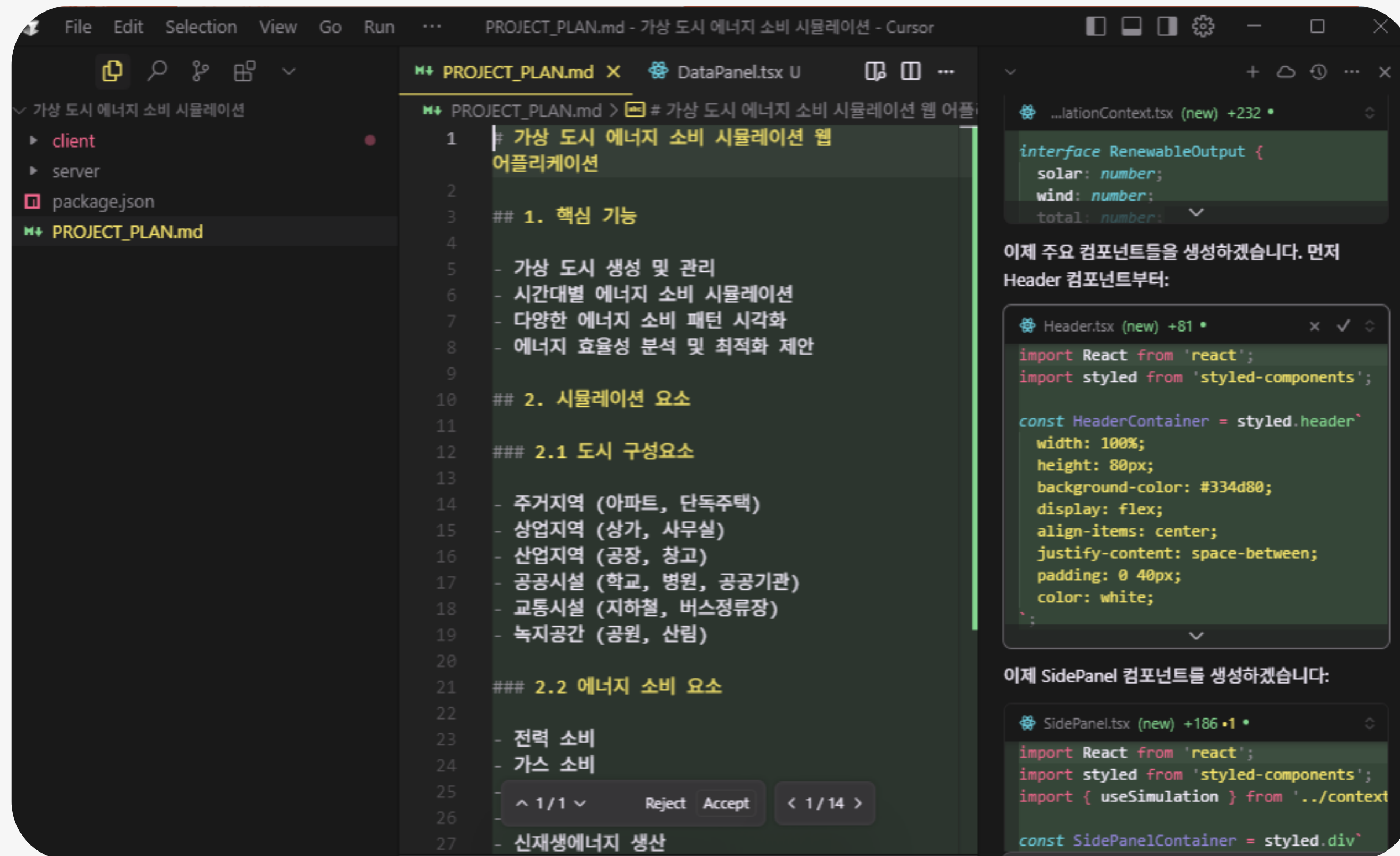
오류 분석

코드 문서화

프로토타이핑

1. 코드 생성 및 자동 완성

원하는 기능, 코드 등의 제작을 요청하면, AI가 이를 분석하고, 그 기능과 현재 상황에 맞는 코드를 자동으로 생성하고 완성할 수 있다.





2. 오류 분석

프로그램에서 발생한 오류를 분석하고, 이를 해결하는 방법을 제시할 수 있다.

또한 사용자의 요청에 따라, 오류를 해결한 코드를 생성하여 기존 코드를 수정하는 것도 가능하다.

The screenshot shows a code editor window with a terminal at the bottom. The terminal output shows the command `cd server && npm start` being executed, followed by `npm start`. The output indicates that the `airmap-korea-server@1.0.0` package is starting, but there are several API errors and a status code 500 error. The sidebar on the right provides a detailed analysis of the errors and suggestions for improvement.

대기오염 예보 웹 어플리케이션 개발

- 대기질 등급 표시

🔥 현재 발생하는 문제

text Apply

측정소 목록 조회 오류: Request failed w

🔧 개선이 필요한 부분

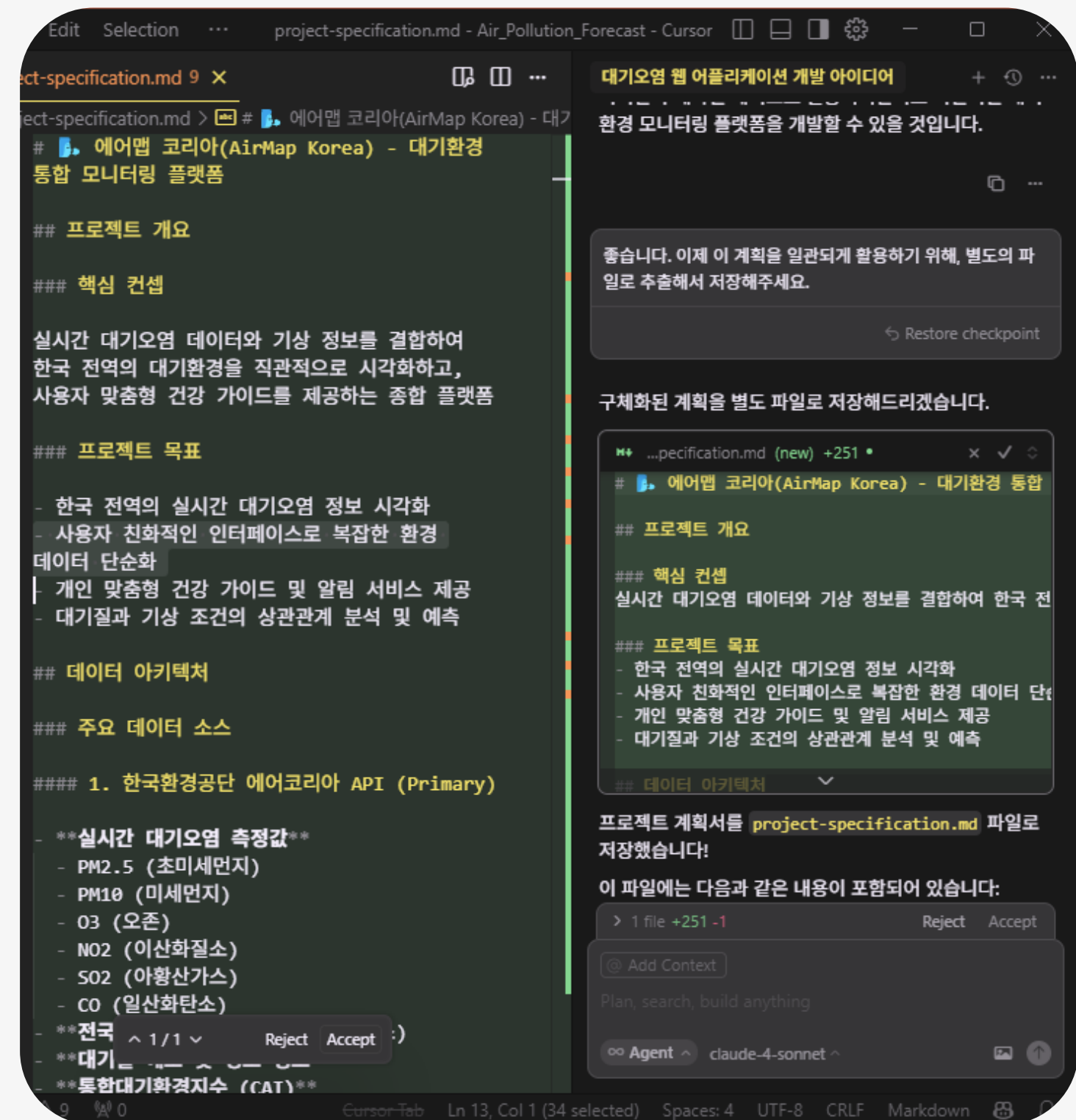
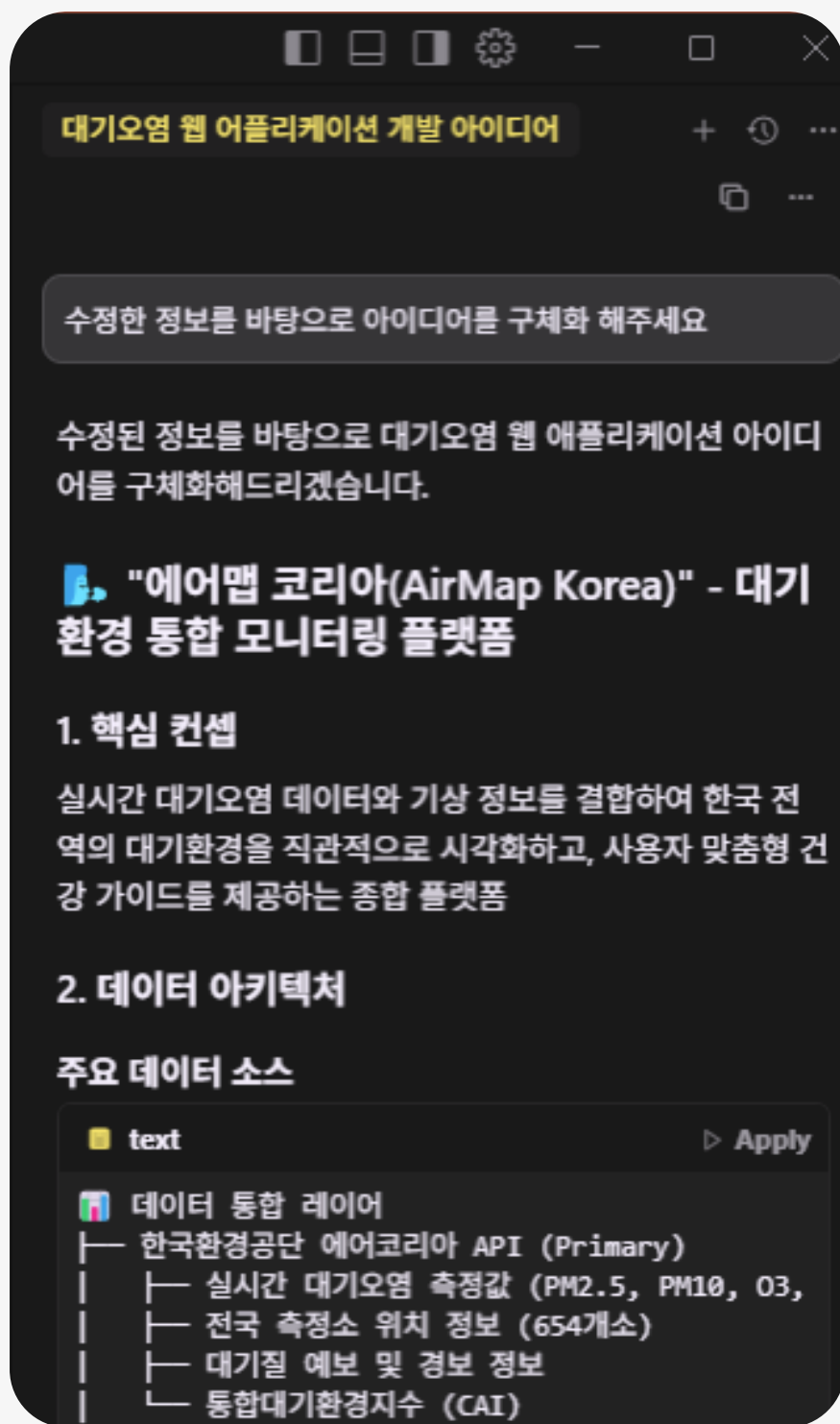
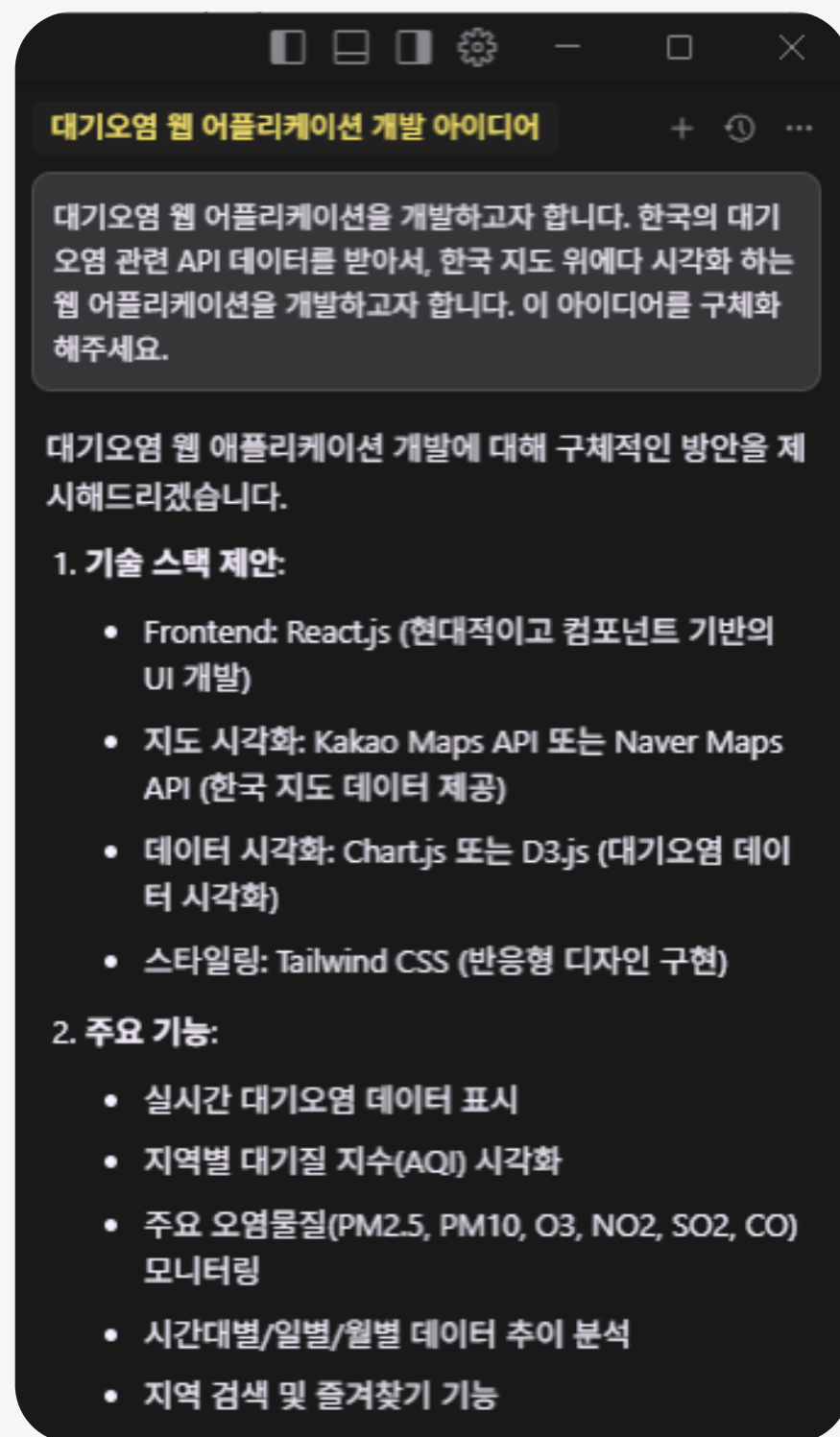
1. **더미 데이터 개선:** 현재 더미 데이터가 너무 단순함
2. **오류 처리 강화:** 500 에러에 대한 구체적인 처리 필요
3. **캐싱 구현:** 측정소 목록은 자주 변경되지 않으므로 캐싱 필요
4. **데이터 검증:** 응답 데이터 구조 검증 강화
5. **지역명 표준화:** 지역명 매핑 테이블 필요

💡 개선 제안사항

어떤 부분을 우선적으로 개선하고 싶으신가요?

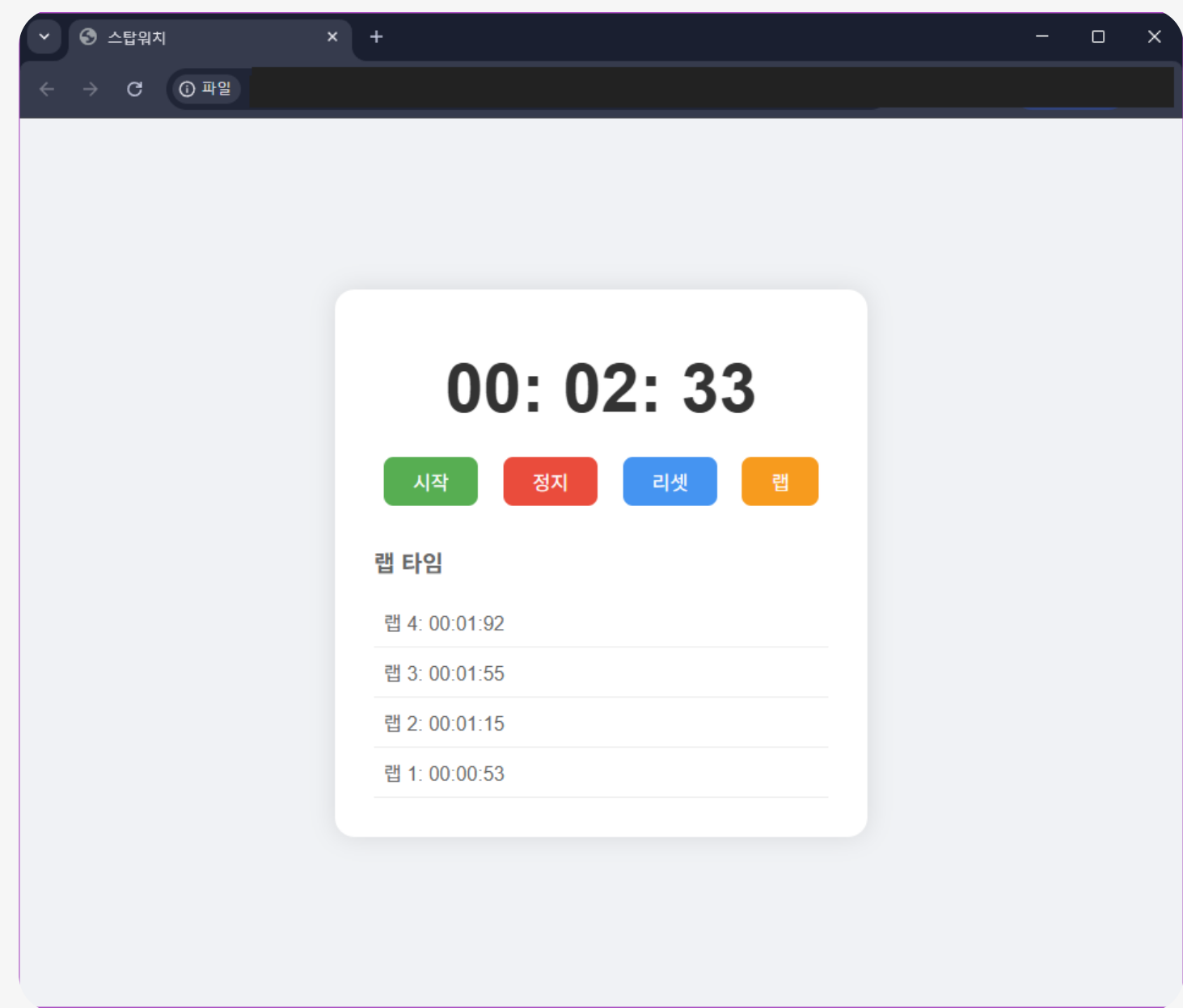
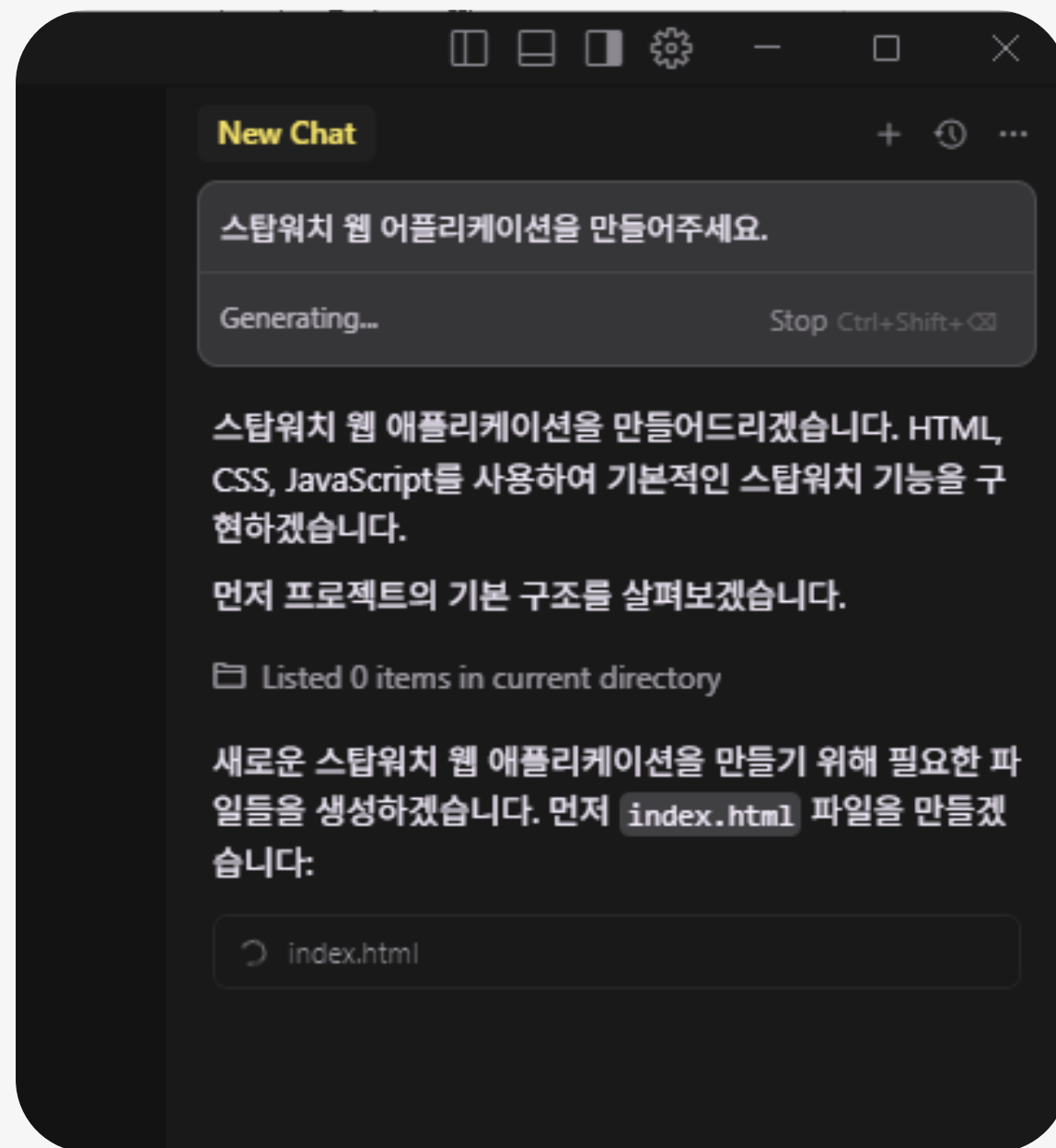
3. 코드 주석 및 문서화

기존 코드베이스를 요약정리해서 위키 문서화 하거나, 사용자의 요구사항을 구체화 하여 별도의 문서로 정리하는 등 새로운/기존 프로젝트를 체계화 하는데 활용할 수 있다.



4. 프로토타이핑

프로젝트의 규모가 작고 단순할 경우, 핵심 기능만 최소한으로 구현한 MVP를 별도의 설계 없이 빠르게 구현하고 테스트 할 수도 있다. 아래 예시의 경우, 프롬프트 하나만 입력하여 실제로 기능하는 스탑워치 웹 어플리케이션을 개발하였다.





Vibe Coding 주의점

코드의 신뢰성

- 생성형 AI는 할루시네이션 현상으로 인해 의도에 맞지 않는 코드, 틀린 코드를 생성할 수 있다.
- 최종 결정권자는 개발자 자신임을 잊지 않고, 생성된 코드를 꼼꼼하게 검수해야 한다.

보안 취약점

- AI가 학습한 코드 데이터 중 보안 면에서 취약한 코드가 있다면, 해당 취약점이 생성된 코드에도 남아 있을 수 있다.
- 코드의 신뢰성 보장과 마찬가지로, 이 역시 개발자가 꼼꼼하게 검수해야 한다.

기술 역량 퇴보 위험

- AI가 생성한 코드를 무비판적으로 수용하다 보면, AI가 생성한 코드를 읽고, 이해하고, 검수할 수 있는 역량이 퇴보할 수 있다.
- 코드를 검수하는 과정에서 코드의 의미, 구조 등이 의도와 설계에 부합하는지 확인할 수 있도록 **기술 역량을 유지하는 것이 좋다.**

Vibe Coding을 통한 생산성 향상의 한계 인지

- 작은 프로젝트에서는 한 개발자가 프로젝트를 위한 모든 요소를 혼자서 개발할 수 있지만, **복잡도가 높거나, 규모가 큰 프로젝트에서는 그렇지 않다.**
- 각자가 Vibe Coding을 통해 생산성을 어느 정도 까지 향상시킬 수 있는지 이해하는 것이 좋다.

Vibe Coding과 개발자의 역할

전통적 개발: 구현자

- 수동으로 코드 작성
- 언어, 구문, 알고리즘에 집중
- 완전한 통제권을 갖고 각 기능 구현

Vibe Coding: AI 관리자

- AI에게 자연어로 의도 전달
- 시스템 구조 설계
- AI 생성물 검증/시스템 통합
- 전략적 문제 해결에 집중